



UNIVERSIDAD NACIONAL DE LA MATANZA



Universidad Nacional de La Matanza

DIIT 3646 – Paradigmas de Programación

Primer Cuatrimestre 2026

Segundo Trabajo Práctico: Magos y Mortifagos

EQUIPO ALPHA

INTEGRANTES

Santino Arazola

Nahuel Barros

Ornella Bonachera

Thiago Dal Secco

Nahuel Palmieri

Franco Perez

PROFESORES

Federico Gasior

Hernan Lanzilotta



Introducción.....	3
Desarrollo y Diseño.....	5
Patrón Composite.....	5
Patrón State.....	5
Factory Method.....	5
Colecciones elegidas.....	5
Conclusión.....	6
Referencias.....	6

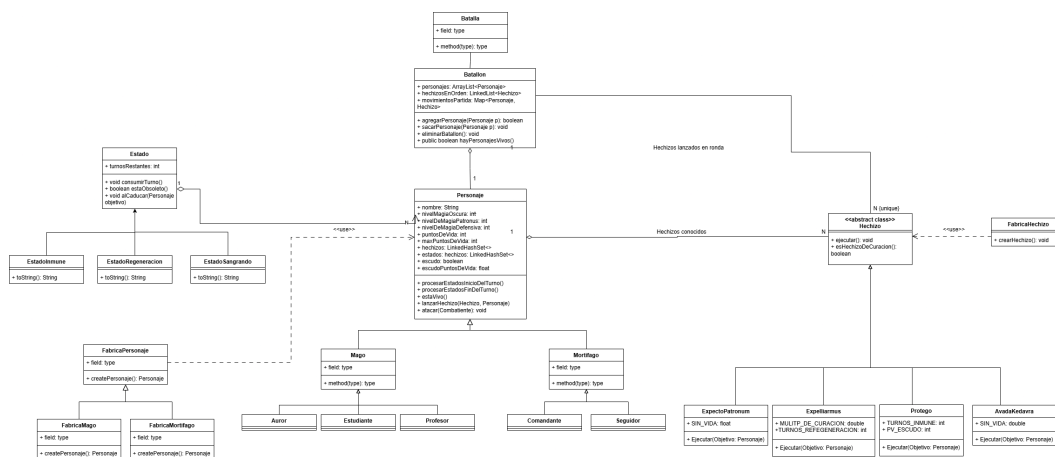
Introducción

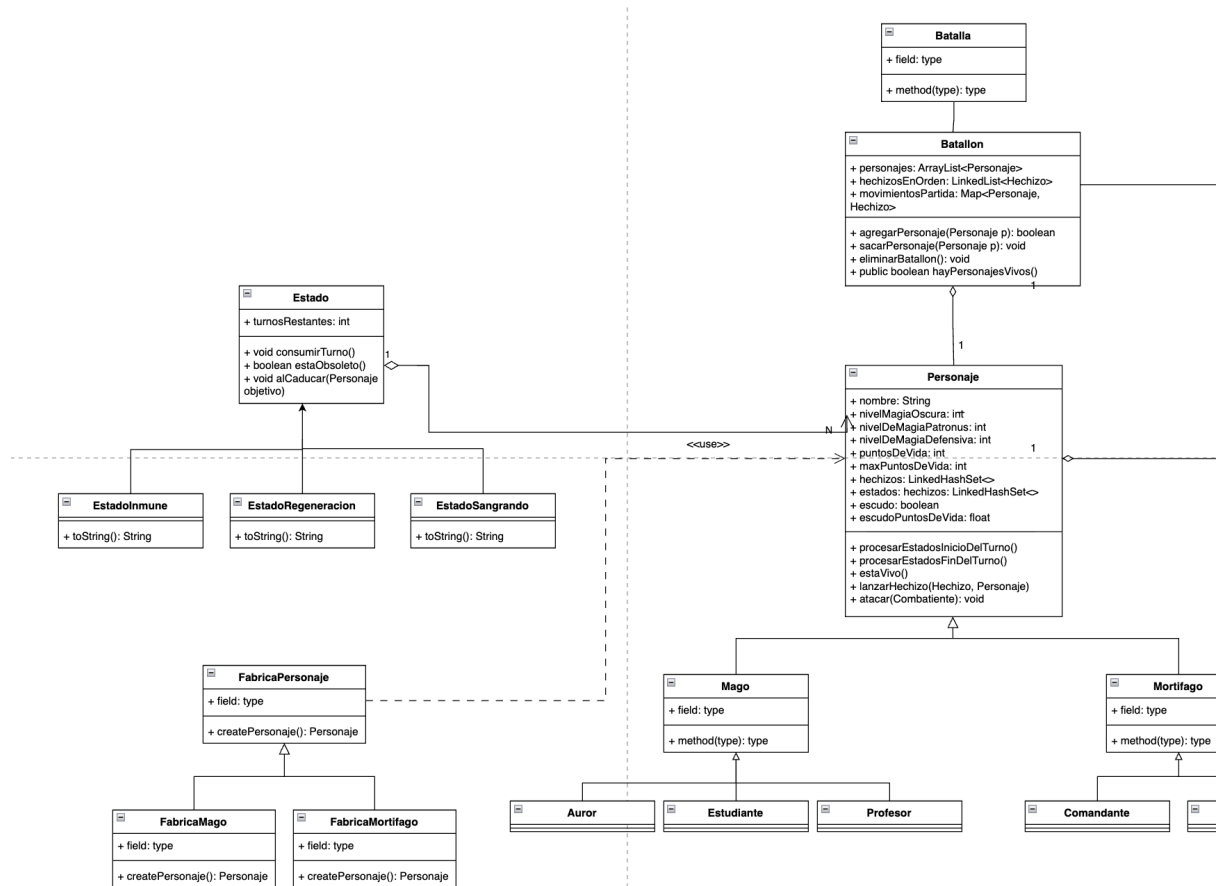
El presente informe detalla el diseño y desarrollo de un juego de simulación de combate. La temática consiste en una batalla automatizada entre dos facciones: Magos y Mortífagos. Cada bando posee características y habilidades particulares, sus turnos de acción y el repertorio de hechizos disponibles. **(Fandom, s.f.)**

Los magos y mortífagos pertenecen a un batallón, en el cual cada personaje elige un hechizo para lanzar por turno, estos hechizos no pueden repetirse por batallón y pueden tener de objetivo a un enemigo o aliado, con la posibilidad de curar, escudar o dañar al objetivo. Además, los hechizos pueden dejar al objetivo en un estado de Inmunidad, Regeneración o Sangrado. **(Universidad Nacional de La Matanza, 2026)**

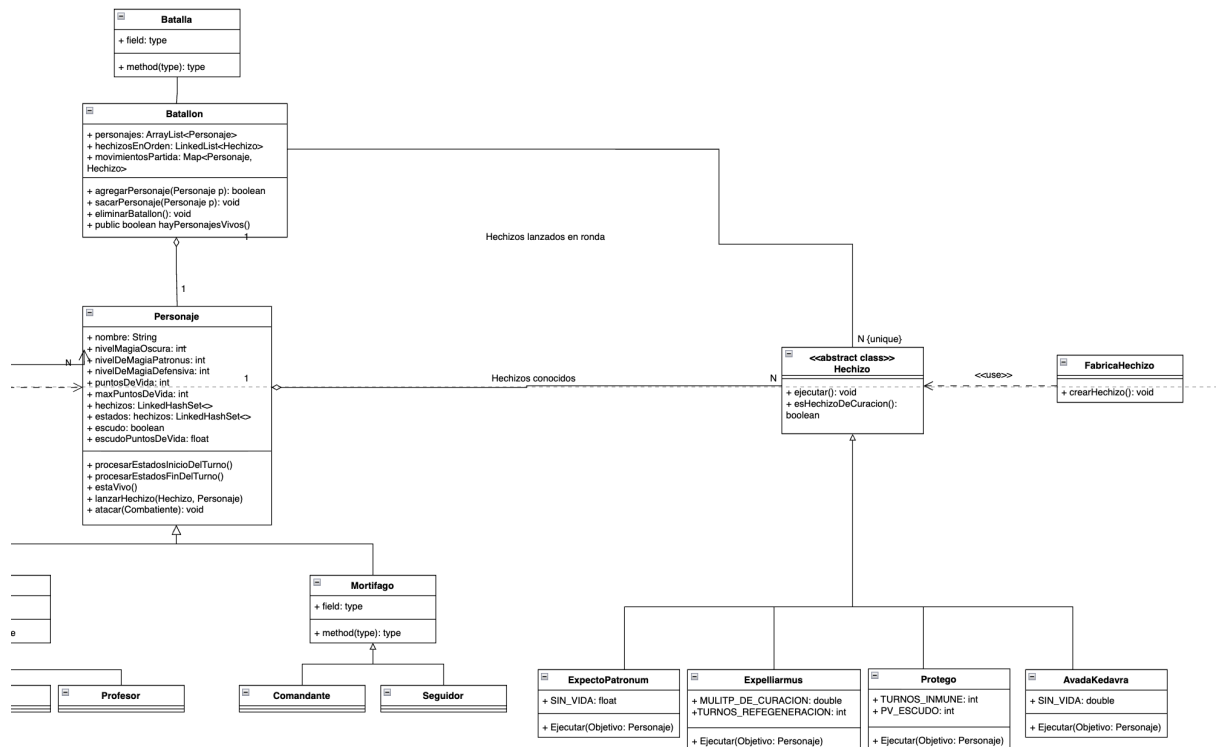
Se eligen aleatoriamente los hechizos con los que se crea cada personaje y luego se elige aleatoriamente los hechizos que van a utilizar por ronda. Los batallones se siguen atacando hasta que todos los miembros de un batallón hayan sido derrotados, el batallón que siga en pie será el ganador.

Hemos creado el siguiente diagrama de clases para representar la estructura de cómo los objetos interactúan entre sí en el sistema.





Parte izquierda



Parte derecha



Desarrollo y Diseño

El diseño del sistema se organiza a partir de cuatro abstracciones centrales y los patrones que las sostienen:

- Combatiente (interfaz): proporciona polimorfismo entre individuos y grupos.
- Personaje (jerarquía abstracta): representa unidades concretas con atributos de combate.
- Hechizo (jerarquía abstracta): acciones encapsuladas. Si bien la consigna especificaba que sea una interfaz, se decidió implementar una clase abstracta para evitar hechizos duplicados en un mismo turno.
- Estado (jerarquía abstracta): modificadores temporales del comportamiento del personaje.

Patrón State (Shvets, 2026).

Clases: Hechizos / Personaje / Estado.

Un personaje puede estar en simultáneo en estado inmune, sangrando regeneración. Cada personaje puede reaccionar de forma distinta ante el daño realizado por un hechizo determinado dependiendo del estado en el que se encuentre. El patrón State encapsula cada modo en una clase y delega el comportamiento a la misma.

Con estos fines se tiene la clase Estado, que define el contrato del estado y los datos comunes. Luego se tienen las clases concretas que heredan de estado: EstadoInmune, EstadoSangrando y EstadoRegeneracion. Los estados concretos pueden ejecutar su efecto al principio o al final del turno, para eso exponen 2 metodos:

aplicarAlInicioDelTurno(Personaje) y aplicarAlFinalDelTurno(Personaje).

Factory Method (Shvets, 2026).

Se utilizó el patrón factory method para la creación de Hechizos y Personajes.

La clase FabricaDeHechizos tiene un método llamado crearHechizo() que realiza un switch sobre un número aleatorio devuelve una de las cuatro subclases de Hechizo.

Por otro lado, la FabricaDePersonajes expone dos métodos (crearMago, crearMortifago) que devuelven subclases aleatorias dentro de su bando. Añade una garantía de unicidad por nombre mediante un Set que guarda los personajes que creó la fábrica, para evitar la generación de duplicados.

Colecciones elegidas

LinkedHashSet<Hechizo> y LinkedHashSet<Estado> **Oracle. (2026).**: conservan orden de inserción → iteración determinista en combate y en los logs. Nos decantamos por no utilizar Lista ya que debíamos verificar que un personaje no tenga hechizos repetidos.



Set<Estado> con equals/hashCode por clase **Oracle. (2026)**: impide acumular instancias duplicadas del mismo tipo de estado.

Conclusión

Luego de implementar los patrones y completar el proyecto concluimos que el desarrollo de este simulador de combate fue mucho más fácil aplicando modularidad, ya que cada persona se centró en una funcionalidad y cómo interactuaba con las otras, además cambiar código por lógica que habíamos pensado mal fue muy fácil ya que cada clase se ocupaba de un propósito y no tuvimos que modificar múltiples archivos. Los test unitarios nos ayudaron a comprobar que los métodos con lógica de negocio funcionen de manera correcta e interactúen bien entre clases. En general se evidenció que el éxito de un software no radica únicamente en cumplir con la lógica funcional, sino en la correcta elección de su arquitectura interna. La combinación de los principios de la POO, la rigurosidad de los patrones GoF y el respaldo de las pruebas unitarias automatizadas permitieron transformar un problema complejo de reglas interdependientes en un sistema robusto, elegante, testeable y preparado para escalar ante futuros requerimientos.

Referencias

- **Oracle. (2026).** *Java Platform, Standard Edition (Java SE) Documentation*. Oracle Help Center. <https://docs.oracle.com/en/java/>
- **Shvets, A. (2026).** *Design Patterns: Creational Patterns (Factory Method)*. Refactoring.Guru. <https://refactoring.guru/design-patterns/creational-patterns>
- **Universidad Nacional de La Matanza. (2026).** *Material de estudio de la asignatura Paradigmas de Programación*. Plataforma MleL. <https://miel.unlam.edu.ar/>
- **Fandom. (s.f.).** *Mortífago*. Wiki Harry Potter. <https://harrypotter.fandom.com/es/wiki/Mort%C3%ADfago>
- **Fandom. (s.f.).** *Mago*. Wiki Harry Potter. <https://harrypotter.fandom.com/es/wiki/Mago>